

# Bambu Cloud / Tidbyt Setup Notes

Updated June 15, 2026.

## Overview

This setup uses a Python polling script to query the Bambu Lab cloud API once per minute and write a local `status.json` file for the Tidbyt app to read. The script now includes persistent idle tracking plus approximate live progress and remaining time derived from task metadata.  
[cite:773][cite:776]

## Current data model

Each printer entry in `status.json` includes core identity and state fields such as `id`, `name`, `online`, `state`, `cloud_print_status`, and `cloud_print_job`.

It also includes print metadata such as `job_name`, `job_objects`, `estimated_duration_seconds`, and `estimated_duration_text`, which come from the cloud task record and its `costTime` value.  
[cite:773]

## Persistent idle tracking

The polling script now persists printer state across runs by reading the previous `status.json` before writing the new one.

This allows the Tidbyt app to show a real idle duration after a print finishes instead of losing that context on every cron run.

## New persistent fields

Each printer entry may include:

- `idle_since`: UTC ISO timestamp for when the printer most recently entered idle after a detected print completion. If the script has not yet observed that transition since deployment or reboot, this stays `null`.
- `last_completed_job_id`: the most recently completed cloud job id known to the script.
- `last_completed_job_name`: the print title for the last completed job.
- `last_completed_job_objects`: the object/design title for the last completed job.
- `last_completed_end_time`: cloud-reported end time when available, otherwise the observed transition time.
- `last_completed_duration_seconds`: estimated total duration of the last completed print.
- `last_completed_duration_text`: human-friendly duration such as 3h 29m.

## Idle-state behavior

- While printing, `idle_since` is cleared and the active job information is shown.
- When the script sees a transition from PRINTING to non-printing, it records `idle_since` and stores that job as the last completed job.
- If the service starts while the printer is already idle, the script does not guess. In that case `idle_since` remains null until a real completion transition is observed later.

That means the Tidbyt app should display ? for idle time when `idle_since` is unknown, instead of pretending it knows the answer.

## Approximate progress and time left

The Bambu Lab cloud endpoints currently used here do not expose live print progress or live remaining time directly. The `bind` endpoint returns high-level device status such as `online`, `print_status`, and `print_job`, while the `tasks` endpoint returns task metadata such as `startTime`, `endTime`, and `costTime`.<sup>[cite:773]</sup><sup>[cite:776]</sup>

To restore a useful live display, the script now computes approximate progress and remaining time for an active print using:

- `startTime` from the current task,
- `costTime` from the current task as the planned total duration,
- the current UTC time at script runtime.<sup>[cite:773]</sup>

## Estimated fields

- `progress`: integer percentage from 0 to 100, estimated from elapsed time divided by planned duration.
- `time_left`: integer seconds remaining, estimated from planned duration minus elapsed time.

These values are estimates, not direct live telemetry from the printer. They may drift if the print speeds up, slows down, pauses, or behaves differently from the planned slicer time.<sup>[cite:780]</sup><sup>[cite:781]</sup>

## Current API behavior

The cloud data observed in this setup supports these conclusions:

| Endpoint                                   | Observed fields                                                                                                 | Limitation                                                                                                                      |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>/v1/iot-service/api/user/bind</code> | <code>online</code> , <code>print_status</code> , <code>print_job</code> , model info <sup>[cite:773]</sup>     | No live percentage or live remaining time <sup>[cite:773]</sup>                                                                 |
| <code>/v1/user-service/my/tasks</code>     | <code>startTime</code> , <code>endTime</code> , <code>costTime</code> , titles, device id <sup>[cite:773]</sup> | Contains planned duration and task history, not direct real-time progress telemetry <sup>[cite:773]</sup> <sup>[cite:776]</sup> |

## Install updated script

Overwrite the existing script:

```
cat <<'PYEOF' > /home/rjison9000/bambu-cloud/cloud_status_update.py
#!/usr/bin/env python3
import json
from datetime import datetime, timezone
from pathlib import Path

import requests

STATUS_PATH = Path("/home/rjison9000/bambu-cloud/status.json")
TOKEN_PATH = Path("/home/rjison9000/.bambu_token")
API_BASE = "https://api.bambulab.com"

def iso_now():
    return datetime.now(timezone.utc).replace(microsecond=0).isoformat()

def read_token():
    raw = TOKEN_PATH.read_text().strip()
    if raw.startswith("{"):
        try:
            data = json.loads(raw)
            t = data.get("token") or data.get("access_token")
            if t:
                return t.strip()
        except Exception:
            pass
    return raw

def api_get(path, token, params=None):
    url = f"{API_BASE}{path}"
    headers = {
        "Authorization": f"Bearer {token}",
        "Accept": "application/json",
    }
    resp = requests.get(url, headers=headers, params=params, timeout=15)
    resp.raise_for_status()
    return resp.json()

def fmt_duration(seconds):
    if seconds is None:
        return None
    try:
        seconds = int(seconds)
    except Exception:
        return None
    if seconds < 60:
        return f"{seconds}s"
    minutes, _ = divmod(seconds, 60)
    if minutes < 60:
```

```

        return f"{minutes}m"
    hours, minutes = divmod(minutes, 60)
    if minutes == 0:
        return f"{hours}h"
    return f"{hours}h {minutes}m"

def normalize_state(online, raw_status, cloud_print_job):
    if not online:
        return "OFFLINE"
    status = (raw_status or "").upper()
    if status in ("RUNNING", "PRINTING"):
        return "PRINTING"
    if status in ("PAUSE", "PAUSED"):
        return "PAUSED"
    if status in ("SUCCESS", "FINISH", "COMPLETED", "IDLE", ""):
        return "IDLE"
    if cloud_print_job:
        return "IDLE"
    return status

def list_printers(token):
    data = api_get("/v1/iot-service/api/user/bind", token)
    return data.get("devices") or data.get("data") or []

def fetch_tasks(token):
    data = api_get("/v1/user-service/my/tasks", token)
    hits = data.get("hits")
    return hits if isinstance(hits, list) else []

def index_tasks_by_device_and_id(tasks):
    tasks_by_id = {}
    tasks_by_device = {}
    for t in tasks:
        if not isinstance(t, dict):
            continue
        device_id = t.get("deviceId")
        job_id = t.get("id")
        if not device_id:
            continue
        if job_id is not None:
            try:
                tasks_by_id[(str(device_id), int(job_id))] = t
            except Exception:
                pass
        tasks_by_device.setdefault(str(device_id), []).append(t)
    for dev_id, lst in tasks_by_device.items():
        lst.sort(key=lambda task: task.get("startTime") or "", reverse=True)
    return tasks_by_id, tasks_by_device

def pick_task_for_printer(device_id, cloud_print_job, tasks_by_id, tasks_by_device):
    dev_key = str(device_id)

```

```

    if cloud_print_job is not None:
        try:
            hit = tasks_by_id.get((dev_key, int(cloud_print_job)))
            if hit:
                return hit
        except Exception:
            pass
    dev_tasks = tasks_by_device.get(dev_key) or []
    if dev_tasks:
        return dev_tasks[0]
    return None

def extract_job_fields_from_task(task):
    if not isinstance(task, dict):
        return None, None, None, None, None
    job_name = task.get("title") or task.get("designTitle") or None
    job_objects = task.get("designTitle") or None
    estimated_seconds = task.get("costTime")
    estimated_text = fmt_duration(estimated_seconds)
    end_time = task.get("endTime") or None
    return job_name, job_objects, estimated_seconds, estimated_text, end_time

def load_previous_status():
    if not STATUS_PATH.exists():
        return {}
    try:
        data = json.loads(STATUS_PATH.read_text())
        printers = data.get("printers", [])
        out = {}
        for p in printers:
            pid = p.get("id")
            if pid:
                out[str(pid)] = p
        return out
    except Exception:
        return {}

def build_printer_record(device, task, previous, now_iso):
    printer_id = str(device.get("dev_id") or device.get("id") or device.get("sn") or
    printer_name = device.get("name") or device.get("dev_name") or device.get("device")
    online = bool(device.get("online", False))

    raw_status = device.get("print_status") or device.get("status") or device.get("s
    cloud_print_job = device.get("print_job")
    state = normalize_state(online, raw_status, cloud_print_job)

    job_name, job_objects, est_seconds, est_text, task_end_time = extract_job_fields.

    progress = None
    time_left = None
    if state == "PRINTING" and task is not None:
        start_str = task.get("startTime")
        if start_str:

```

```

try:
    ts = start_str.replace("Z", "+00:00")
    start_dt = datetime.fromisoformat(ts)
    now_dt = datetime.now(timezone.utc)
    elapsed = max((now_dt - start_dt).total_seconds(), 0)
    if est_seconds:
        total = max(int(est_seconds), 1)
        progress = min(100, max(0, int(elapsed * 100 // total)))
        time_left = max(total - int(elapsed), 0)
except Exception:
    pass

prev_state = (previous or {}).get("state")
prev_job = (previous or {}).get("cloud_print_job")
prev_idle_since = (previous or {}).get("idle_since")
prev_last_completed_job_id = (previous or {}).get("last_completed_job_id")
prev_last_completed_job_name = (previous or {}).get("last_completed_job_name")
prev_last_completed_job_objects = (previous or {}).get("last_completed_job_objects")
prev_last_completed_end_time = (previous or {}).get("last_completed_end_time")
prev_last_completed_duration_seconds = (previous or {}).get("last_completed_duration_seconds")
prev_last_completed_duration_text = (previous or {}).get("last_completed_duration_text")

idle_since = None
last_completed_job_id = prev_last_completed_job_id
last_completed_job_name = prev_last_completed_job_name
last_completed_job_objects = prev_last_completed_job_objects
last_completed_end_time = prev_last_completed_end_time
last_completed_duration_seconds = prev_last_completed_duration_seconds
last_completed_duration_text = prev_last_completed_duration_text

if state == "PRINTING":
    idle_since = None
else:
    if prev_state == "PRINTING":
        idle_since = now_iso
        last_completed_job_id = cloud_print_job or prev_job
        last_completed_job_name = job_name or prev_last_completed_job_name
        last_completed_job_objects = job_objects or prev_last_completed_job_objects
        last_completed_end_time = task_end_time or now_iso
        last_completed_duration_seconds = est_seconds
        last_completed_duration_text = est_text
    elif prev_state in ("IDLE", "PAUSED", "OFFLINE") and prev_idle_since not in (None, "0000-00-00T00:00:00"):
        idle_since = prev_idle_since
    else:
        idle_since = None
        if last_completed_job_id is None and cloud_print_job is not None:
            last_completed_job_id = cloud_print_job
        if last_completed_job_name is None and job_name is not None:
            last_completed_job_name = job_name
        if last_completed_job_objects is None and job_objects is not None:
            last_completed_job_objects = job_objects
        if last_completed_duration_seconds is None and est_seconds is not None:
            last_completed_duration_seconds = est_seconds
            last_completed_duration_text = est_text

if state == "PRINTING":

```

```

        display_job_name = job_name
        display_job_objects = job_objects
        display_est_seconds = est_seconds
        display_est_text = est_text
    else:
        display_job_name = last_completed_job_name or job_name
        display_job_objects = last_completed_job_objects or job_objects
        display_est_seconds = last_completed_duration_seconds if last_completed_duration_seconds else est_seconds
        display_est_text = last_completed_duration_text or est_text

    return {
        "id": printer_id,
        "name": printer_name,
        "online": online,
        "state": state,
        "label": raw_status,
        "cloud_print_status": raw_status,
        "cloud_print_job": cloud_print_job,
        "job_name": display_job_name,
        "job_objects": display_job_objects,
        "progress": progress,
        "time_left": time_left,
        "estimated_duration_seconds": display_est_seconds,
        "estimated_duration_text": display_est_text,
        "idle_since": idle_since,
        "last_completed_job_id": last_completed_job_id,
        "last_completed_job_name": last_completed_job_name,
        "last_completed_job_objects": last_completed_job_objects,
        "last_completed_end_time": last_completed_end_time,
        "last_completed_duration_seconds": last_completed_duration_seconds,
        "last_completed_duration_text": last_completed_duration_text,
        "probe_errors": [],
    }
}

```

```

def main():
    token = read_token()
    printers_out = []
    top_errors = []
    now_iso = iso_now()
    previous_by_id = load_previous_status()

    try:
        devices = list_printers(token)
    except Exception as e:
        devices = []
        top_errors.append(f"list_printers: {repr(e)}")

    try:
        tasks = fetch_tasks(token)
        tasks_by_id, tasks_by_device = index_tasks_by_device_and_id(tasks)
    except Exception as e:
        tasks_by_id, tasks_by_device = {}, {}
        top_errors.append(f"fetch_tasks: {repr(e)}")

    for device in devices:

```

```

try:
    printer_id = str(device.get("dev_id") or device.get("id") or device.get("name"))
    cloud_print_job = device.get("print_job")
    task = pick_task_for_printer(printer_id, cloud_print_job, tasks_by_id, tasks_by_name)
    previous = previous_by_id.get(printer_id, {})
    printers_out.append(build_printer_record(device, task, previous, now_iso))
except Exception as e:
    printers_out.append({
        "id": str(device.get("dev_id") or "unknown"),
        "name": device.get("name") or "Unknown Printer",
        "online": False,
        "state": "ERROR",
        "label": "ERROR",
        "cloud_print_status": None,
        "cloud_print_job": None,
        "job_name": None,
        "job_objects": None,
        "progress": None,
        "time_left": None,
        "estimated_duration_seconds": None,
        "estimated_duration_text": None,
        "idle_since": None,
        "last_completed_job_id": None,
        "last_completed_job_name": None,
        "last_completed_job_objects": None,
        "last_completed_end_time": None,
        "last_completed_duration_seconds": None,
        "last_completed_duration_text": None,
        "probe_errors": [f"build_printer_record: {repr(e)}"],
    })

payload = {
    "updated_at": now_iso,
    "printers": printers_out,
    "probe_errors": top_errors,
}

STATUS_PATH.write_text(json.dumps(payload, indent=2))
print(f"Wrote {STATUS_PATH} with {len(printers_out)} printers")

if __name__ == "__main__":
    main()
PYEOF

chmod +x /home/rjison9000/bambu-cloud/cloud_status_update.py
python3 -m py_compile /home/rjison9000/bambu-cloud/cloud_status_update.py

```

Manual test run:

```
cd /home/rjison9000/bambu-cloud && /home/rjison9000/bambu-cloud/.venv/bin/python /home/rjison9000/bambu-cloud/cloud_status_update.py
```



## Current observed behavior

A successful current test showed output including:

- `state: "PRINTING"`
- `progress: 26`
- `time_left: 9247`
- `estimated_duration_text: "3h 29m"`
- `idle_since: null`

That is consistent with a job that is in progress and using estimated remaining time rather than direct printer telemetry.[cite:773]

## Cron

No cron schedule change is required. The same once-per-minute cron job can keep running the updated script.[cite:726]

## Local HTTP serving for the JSON

The Tidbyt app reads the generated `status.json` over HTTP from the Raspberry Pi. A quick way to expose that file is Python's built-in HTTP server on port 8787, which serves the current directory and can be kept alive with `nohup`.[cite:752][cite:760][cite:761]

Start it manually:

```
cd /home/rjison9000/bambu-cloud
python3 -m http.server 8787
```

Or keep it running after logout:

```
cd /home/rjison9000/bambu-cloud
nohup python3 -m http.server 8787 > /home/rjison9000/bambu-cloud/http_server.log 2>&
```

The JSON should then be available at `http://192.168.68.84:8787/status.json` on the local network.